```
      create_table :users do |t|
»       t.string :name, null: false
        t.string :email_address, null: false
        t.string :password_digest, null: false

        t.timestamps
      end
      add_index :users, :email_address, unique: true
    end
  end
```

And then run the migration:

```
depot> bin/rails db:migrate
```

Next, we have to flesh out the user model:

rails80/depot_r/app/models/user.rb

```
  class User < ApplicationRecord
»   validates :name, presence: true, uniqueness: true
»   validates :email_address, presence: true, uniqueness: true
    has_secure_password
    has_many :sessions, dependent: :destroy

    normalizes :email_address, with: ->(e) { e.strip.downcase }
  end
```

We check that the name and email addresses are present and unique (that is, no two users can have the same name or email address in the database).

Then there's the mysterious `has_secure_password`.

You know those forms that prompt you to enter a password and then make you reenter it in a separate field so they can validate that you typed what you thought you typed? That's exactly what `has_secure_password` does for you: it tells Rails to validate that the two passwords match.

A user is defined to have many sessions, and those sessions are to be destroyed when the user is destroyed. Finally, email addresses are